Technical Appendix: LLM-Assisted Coding Process

This appendix describes the software process (i.e., pipeline) used to apply the study codebook to interview transcripts with the assistance of large language models. The codebook itself, its development, and the procedures used to review and reconcile model output with human judgment are documented in the main text and in the accompanying methods sections. The purpose of this appendix is to detail the computational steps that produced the model-generated qualitative coding output. Computer-science terms are introduced briefly on first use. A Mac Studio with an M4 chip and 129 GB of unified memory was used.

A large language model (LLM) is a statistical model of natural language trained to predict the next word in a sequence. The mode is further training to perform tasks via reinforcement learning. An open-weights model is an LLM model whose trained parameters are publicly released, so that the model can be downloaded and run on local hardware rather than accessed through a remote service.

A.1 Overview

The pipeline applies a predefined codebook to interview transcripts using locally hosted, open-weights LLMs served through Ollama, an open-source program that runs language models on local hardware and exposes them through a simple programming interface or a Python Application Programming Interface (API). The pipeline is organized in two stages. The first stage assembles a complete instruction for the model, called a prompt, by combining a short role statement, a longer set of coding instructions, the codebook, and the transcript text. The second stage submits each assembled prompt to a chosen model and records the model's reply. Separating the two stages allows the same set of prompts to be passed through several models in succession without rebuilding them, so any comparison between models rests on identical inputs. We used the same parameters for all models.

A.2 Inputs

Three template files are stored in the input/ directory:

- system_prompt.txt: a short instruction establishing the role the model is asked to adopt, namely that of an expert qualitative researcher applying a predefined codebook.
- user_prompt.txt: the main set of coding instructions, including the required output schema and two placeholder markers, <codes> and <transcript>, that indicate where the codebook and a given transcript should be inserted at build time.
- codes.txt: the codebook, treated by the pipeline as a block of text that is copied verbatim into each prompt. The codebook itself is described in the methods section of the paper.
- Interview transcripts are stored as one plain-text file per interview in the transcripts/ directory. Within each transcript, interviewer questions and comments are explicitly labeled at the source, so that the instructions can ask the model to exclude them from coding. We include a synthetic (simulated) interview describing the structure of the transcripts.

File reading accepts two common text encodings, UTF-8 and Windows-1252, which accommodates curly quotation marks and other characters introduced by some transcription tools without requiring manual cleanup.

A.3 Prompt construction

Prompt construction is implemented by build_prompts.py. For each transcript, the script substitutes the codebook into the <codes> marker and the transcript text into the <transcript> marker of the user-prompt template, producing one composite prompt file per transcript in the prompts/ directory. The shared role statement is written once to the same directory and reused.

The substitution is performed using a pattern-matching procedure (a regular expression, a standard mechanism for locating text that matches a defined shape). The procedure raises an error and halts if either marker is missing from the template, so that an editing mistake to the template is detected at the time prompts are built rather than later, during model inference, when it would be more costly to discover.

The user-prompt template encodes the substantive coding instructions that the model is asked to follow:

- Each coded passage must be reproduced verbatim from the transcript, with no paraphrase, abbreviation, or omission.
- Codes are to be applied conservatively. The instructions state that under-coding is preferable to over-coding when evidence is ambiguous.
- Each code assignment must be supported by an explicit quotation from the passage.
- Interviewer turns must not be coded.
- Multiple codes may be assigned to a single passage only when each is independently supported by the text.

For every coded passage, the model is asked to return a structured record containing six fields: the verbatim paragraph, the primary code, any secondary codes, a justification, a supporting quotation for each code applied, and a self-reported confidence rating of low, medium, or high. After the body of the transcript has been coded, the model is asked to add a short summary of the most frequent codes encountered and a list of passages that the model considered ambiguous. These two summary requests support the human review and reconciliation step described in the methods section.

A.4 Inference

The inference stage is implemented by run_coding.py and uses Ollama as the runtime. No transcript content was transmitted to any third-party service at any stage of the analysis; the rationale for this restriction, and the consequences for model selection, are discussed in A.5.

Before the main loop begins, two preparatory checks are performed. First, the script queries Ollama to confirm that the configured model is installed on the local machine and provides a remediation hint if it is not, so that a configuration error is caught immediately rather than after several minutes of failed inferences. Second, the script compares the modification times of the source files and the previously built prompts and prints a warning if any source file has been edited more recently than the oldest built prompt. The warning reminds the operator to rebuild the prompts if the codebook or any transcript has changed, but it does not abort the run.

The main loop iterates over the prompt files. Each prompt is submitted to the model as a single exchange consisting of the shared role statement and the prompt body, and the model's reply is written to a file under output/<model>/<transcript>.txt. By default, transcripts whose output file already exists for the active model are skipped, which allows a run to be interrupted and resumed without redoing completed work. Errors encountered on a single transcript are written to a separate file with the suffix .ERROR.txt

and the loop continues with the next transcript, so that a transient problem affecting one transcript does not interrupt processing of the remainder.

A.5 Models and sampling parameters

The analysis used two open-weights models served locally: gpt-oss:120b and gemma4:31b. Each transcript was coded by both models, with identical prompts and identical sampling parameters, in successive passes of the inference stage. Differences between the two models' outputs were resolved during the human review and reconciliation step described in the methods section.

Four additional locally hosted models were tried but were not retained for analysis: llama3.3:70b, qwen3.5:122b, deepseek-r1:70b, and mixtral:8x22b. Each prompt combines the role statement, the coding instructions, the codebook, and a transcript, and the longest combined prompt approached the upper limit of what the available hardware could supply in working memory for a model of this size. The four models named could not reliably accommodate the required context window, the maximum amount of text the model is permitted to consider in a single input, at the size needed to fit the largest transcripts.

Two cloud-served open-weights models, glm-5.1 and kimi-k2.6, were considered and rejected on confidentiality grounds. Running these models would have required transmitting interview content to a third-party service. The Ollama terms of services are ambiguous about data retention policies for cloud-hosted models. All inference reported in the paper was performed on hardware under the direct control of the research team.

Table A.1. Inference parameters used for both reported models.

Parameter	Value	Brief description
temperature	0.8	Controls how strongly the model favors its single most likely next word over plausible alternatives; lower values produce more deterministic output.
top_p	0.9	Restricts the model's choice at each step to the smallest set of next-word candidates whose combined probability reaches 0.9. Standard name: *nucleus sampling*.
top_k	64	An additional cap that limits the model's choice at each step to the 64 most probable next-word candidates.
num_ctx	33000	The size of the context window, expressed in tokens. A *token* is a unit of text used by the model; for English prose, one token corresponds to roughly three quarters of a word.
seed	42	A fixed starting value for the model's internal random-number generator. Holding the seed constant produces the same output on repeated runs of the same model.

A fixed random seed yields reproducible output for a given combination of model file and Ollama runtime version. Reproducibility does not extend across model updates, runtime upgrades, or different hardware configurations; this limit is intrinsic to current LLM deployment.

Each output file begins with a metadata header recording the transcript identifier, the date and time of the run in the ISO-8601 calendar-and-clock format (an international standard for representing dates and times, for example 2026-06-05T14:32:10), the elapsed inference time in seconds, and the complete set of

sampling parameters serialized in JSON, a plain-text format for structured data that is both human-readable and amenable to subsequent automated analysis. The header is followed by the model's reply.

Table A.2. Models passed through the pipeline.

Model identifier	Hosting	Used for the reported analysis	Notes
gpt-oss:120b	Local	Yes	
gemma4:31b	Local	Yes	
llama3.3:70b	Local	No	Required context window not reliably supported on the available hardware.
qwen3.5:122b	Local	No	Required context window not reliably supported on the available hardware.
deepseek-r1:70b	Local	No	Required context window not reliably supported on the available hardware.
mixtral:8x22b	Local	No	Required context window not reliably supported on the available hardware.
glm-5.1	Cloud	No	Not used on transcript data for confidentiality reasons.
kimi-k2.6	Cloud	No	Not used on transcript data for confidentiality reasons.

A.7 Repository

The pipeline scripts (config.py, build_prompts.py, run_coding.py), the prompt templates in input/, and the codebook in input/codes.txt are available in the accompanying GitHub repository. The interview transcripts (transcripts/) and the model outputs (output/) are withheld to preserve participant confidentiality. A reader with access to suitable hardware and a local installation of Ollama can install the named models, run the two scripts in sequence, and reproduce the inference step on transcripts of their own.